Parallel Mater

API Specification · 0.2

This specification defines the Parallel Mater 0.2 command API: object ownership, device discovery, explicit storage, physics pipelines, command execution, and synchronization. It is the behavioral contract for implementations and applications. The companion reference guide lists declarations and points here by section number.

Requirements stated as must, must not, required, or an imperative define valid usage and required implementation behavior. Examples illustrate those requirements and do not override them. The core domains and optional extension boundaries are specified in section 15.6; the API-family overview is not a claim that every device supports every physics domain.

Contents

1 Parallel Mater 2

2 Return codes 2

3 Function Pointers and Instancing 3

4 Devices & queues 4

5 Explicit device memory 6

6 Buffers & triangle geometry 7

7 Simulation state & rigid bodies 9

8 Rigid constraints 12

9 Physics pipelines & materials 14

10 Command recording & lifecycle 16

11 Data movement & output 18

12 Submission & synchronization 20

13 Native API interoperability 21

14 Timing, work & memory queries 26

15 Features, limits & formats 28

16 A frame, recorded explicitly 30

17 Fundamental declarations 32

A Symbol index 33

1 Parallel Mater

API 0.2

1.1 Overview

Parallel Mater is a physics API offering functions and structures that allow you to define physics simulations. Parallel Mater supports Rigid Bodies, Fluid, Cloth, Rope, Smoke, and Soft Bodies. Parallel Mater uses a configured fixed-function pipeline to simulate the specified physics simulations and their specified couplings.

1.2 Color key

Function names: pmCreateInstance
Structure and type names: PmInstanceCreateInfo
Constants and return codes: PM_SUCCESS
PFN_pmName: pointer type for pmName
[n.n] Specification section

1.3 Calling conventions

pmCreate*/pmDestroy* own objects; pmGet* queries; pmCmd* records work. Counts precede arrays. Input pointers are const. A null array requests an enumeration count; a short array returns PM_INCOMPLETE and the number written. Unless stated otherwise, input data is copied before a call returns.

1.4 Validity & availability

Set sType to its matching PM_STRUCTURE_TYPE_* value; pNext = NULL in core 0.2. Unknown chains and reserved flags are errors. Initialize required nonzero fields. Null handles are allowed only where specified. Physics domains and couplings require their corresponding features or extensions. This reference specifies the behavior required of conforming implementations.

1.5 Units & lifetime

Right-handed; +Y up; meters, seconds, kilograms, radians, kelvin. Quaternions: normalized x,y,z,w, local-to-world. No implicit coordinate conversion. Device children share one device. Destroy in reverse dependency order after GPU completion; destruction does not wait. Command buffers borrow resources. Externally synchronize host access to queues, pools, and mutable objects.

2 Return codes

Results and failures

2.1 Success & error codes

```
typedef enum PmResult {
    PM_SUCCESS = 0,
    PM_NOT_READY = 1,
    PM_TIMEOUT = 2,
    PM_INCOMPLETE = 3,
    PM_ERROR_INVALID_ARGUMENT = -1,
    PM_ERROR_INVALID_STATE = -2,
    PM_ERROR_OUT_OF_HOST_MEMORY = -3,
    PM_ERROR_OUT_OF_DEVICE_MEMORY = -4,
    PM_ERROR_FEATURE_NOT_PRESENT = -5,
    PM_ERROR_FORMAT_NOT_SUPPORTED = -6,
    PM_ERROR_CAPACITY_EXCEEDED = -7,
    PM_ERROR_DEVICE_LOST = -8,
    PM_ERROR_INCOMPATIBLE_VERSION = -9
} PmResult;
```

Nonnegative codes report success or status; negative codes report failure. Creation clears the output handle before attempting work. Failed creation owns nothing. A failed recording call latches an error; End fails and the

buffer must be reset. Runtime capacity overflow is reported in the step statistics; it is not silently successful physics.

3 Function Pointers and Instancing

Resolve entry points and create an instance

3.1 Function pointers

```
typedef void (PMAPI_CALL *PFN_pmVoidFunction)(
    void);

PFN_pmVoidFunction pmGetInstanceProcAddr(
    PmInstance instance, const char* pName);
PFN_pmVoidFunction pmGetDeviceProcAddr(
    PmDevice device, const char* pName);
```

Every command has a matching PFN_pmCommandName typedef in the header. Cast a returned pointer to that command's exact type before calling it; never call through PFN_pmVoidFunction. pName is an exact, case-sensitive, null-terminated command name. Unknown or unavailable names return NULL.

With instance = NULL, pmGetInstanceProcAddr resolves only itself, pmEnumerateInstanceVersion, and pmCreateInstance. With a live instance it also resolves core instance, physical-device and device commands. pmGetDeviceProcAddr requires a live device and returns only device commands supported by its enabled features and extensions. A non-NULL pointer does not enable a feature.

Obtain pmGetInstanceProcAddr by linking or the platform's library-symbol lookup. Instance-resolved pointers are valid only for that instance and its children; device-resolved pointers only for that device and its children. Keep their owner and the library alive while using them.

Related sections: [3.2], [4.5]

3.2 Instance creation

```
typedef struct PmInstanceCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t apiVersion;
    const char* pApplicationName;
    PmBool32 enableValidation;
} PmInstanceCreateInfo;

PmResult pmEnumerateInstanceVersion(
    uint32_t* pApiVersion);
PmResult pmCreateInstance(
    const PmInstanceCreateInfo* pInfo,
    const PmAllocationCallbacks* pAllocator,
    PmInstance* pInstance);
void pmDestroyInstance(PmInstance instance);
```

pmEnumerateInstanceVersion reports the highest supported API version; pApiVersion must be non-NULL. Set apiVersion = PM_API_VERSION_0_2 to use this reference. Unsupported versions return PM_ERROR_INCOMPATIBLE_VERSION. Destroy every logical device before its instance. Validation is a host-side diagnostic request, not permission to omit required synchronization.

Related sections: [3.3], [4]

3.3 Host allocation callbacks

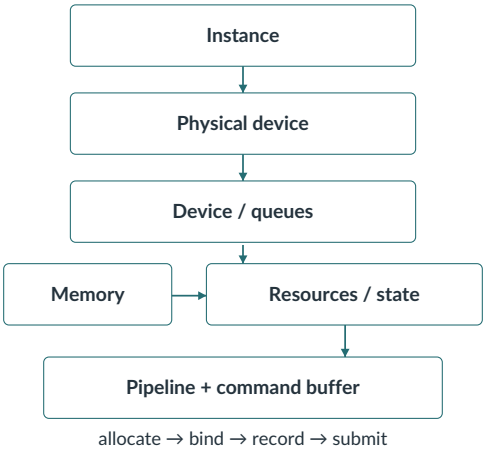
```
typedef void* (PMAPI_CALL *PmAllocateFn)(
    void* user, size_t size, size_t alignment);
typedef void (PMAPI_CALL *PmFreeFn)(
    void* user, void* allocation);
typedef struct PmAllocationCallbacks {
    void* pUserData;
    PmAllocateFn pfnAllocate;
    PmFreeFn pfnFree;
} PmAllocationCallbacks;
```

Callbacks allocate CPU bookkeeping only, with power-of-two alignment. Both functions are required if callbacks are supplied. NULL selects the library host allocator. The instance copies callbacks; user data must survive the instance and all children. This does not allocate device physics storage.

Related sections: [5]

4 Devices & queues

Discover support before creating resources



4.1 Physical devices

```
PmResult pmEnumeratePhysicalDevices(
    PmInstance instance, uint32_t* pCount,
    PmPhysicalDevice* pDevices);
```

Physical-device handles are borrowed from the instance. Pass pDevices = NULL to query the count, then provide storage and enumerate. If the list grows, PM_INCOMPLETE returns the number written; query the count again before retrying.

4.2 Physical-device identity

```
typedef struct PmPhysicalDeviceProperties {
    uint32_t apiVersion;
    uint32_t driverVersion;
    uint32_t vendorId;
    uint32_t deviceId;
    uint8_t deviceUuid[16];
    char deviceName[256];
} PmPhysicalDeviceProperties;

void pmGetPhysicalDeviceProperties(
    PmPhysicalDevice physicalDevice,
    PmPhysicalDeviceProperties* pProperties);
```

Use the UUID to match a renderer's adapter. Names and enumeration indices are not stable identities. driverVersion is implementation-defined; do not decode it as the API version.

Related sections: [15], [13]

4.3 Queue families

```
enum {
    PM_QUEUE_PHYSICS_BIT = 0x1,
    PM_QUEUE_TRANSFER_BIT = 0x2
};
typedef struct PmQueueFamilyProperties {
    PmFlags queueFlags;
    uint32_t queueCount;
    uint32_t timestampValidBits;
    float timestampPeriodNanoseconds;
} PmQueueFamilyProperties;

PmResult pmGetPhysicalDeviceQueueFamilies(
    PmPhysicalDevice physicalDevice,
    uint32_t* pCount,
    PmQueueFamilyProperties* pProperties);
```

PHYSICS queues also support transfer. A zero timestampValidBits means timestamps are unavailable on that family. Queues execute asynchronously; submission order alone is not a memory dependency.

Related sections: [12], [14]

4.4 Supported vs enabled features

```
typedef struct PmPhysicalDeviceFeatures {
    PmBool32 rigidBodies;
    PmBool32 rigidConstraints;
    PmBool32 particleLiquids;
    PmBool32 liquidRigidCoupling;
    PmBool32 liquidLiquidCoupling;
    PmBool32 timelineSemaphores;
    PmBool32 timestampQueries;
    PmBool32 statisticsQueries;
    PmBool32 memoryBudget;
    PmBool32 vulkanInterop;
} PmPhysicalDeviceFeatures;

void pmGetPhysicalDeviceFeatures(
    PmPhysicalDevice physicalDevice,
    PmPhysicalDeviceFeatures* pFeatures);
```

Supported features are not automatically enabled. Request only needed features at device creation. Constraints require rigidBodies; each coupling requires its participating domains. Timeline semaphores are required for a conforming command-API device; other features may be false.

4.5 Logical device

```
typedef struct PmDeviceQueueCreateInfo {
    uint32_t queueFamilyIndex;
    uint32_t queueCount;
    const float* pPriorities;
} PmDeviceQueueCreateInfo;
typedef struct PmDeviceCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t queueCreateInfoCount;
    const PmDeviceQueueCreateInfo* pQueueInfos;
    PmPhysicalDeviceFeatures enabledFeatures;
} PmDeviceCreateInfo;

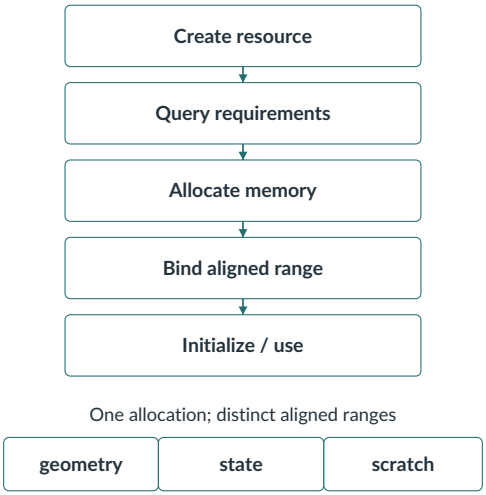
PmResult pmCreateDevice(
    PmPhysicalDevice physicalDevice,
    const PmDeviceCreateInfo* pInfo,
    PmDevice* pDevice);
void pmDestroyDevice(PmDevice device);
void pmGetDeviceQueue(PmDevice device,
    uint32_t family, uint32_t index,
```

```
    PmQueue* pQueue);
```

Request each family at most once; priorities are finite in [0,1]. Unsupported features fail creation. Queues are borrowed, not destroyed separately. Wait for work and destroy children before destroying the device.

5 Explicit device memory

Create → requirements → allocate → bind → use



5.1 Heaps & types

```
enum {
    PM_MEMORY_DEVICE_LOCAL_BIT = 0x1,
    PM_MEMORY_HOST_VISIBLE_BIT = 0x2,
    PM_MEMORY_HOST_COHERENT_BIT = 0x4,
    PM_MEMORY_HOST_CACHED_BIT = 0x8
};
typedef struct PmMemoryHeap {
    PmDeviceSize size;
    PmFlags flags;
} PmMemoryHeap;
typedef struct PmMemoryType {
    PmFlags propertyFlags;
    uint32_t heapIndex;
} PmMemoryType;
typedef struct PmMemoryProperties {
    uint32_t heapCount;
    PmMemoryHeap heaps[16];
    uint32_t typeCount;
    PmMemoryType types[32];
} PmMemoryProperties;

void pmGetPhysicalDeviceMemoryProperties(
    PmPhysicalDevice physicalDevice,
    PmMemoryProperties* pProperties);
```

Heap flags are reserved zero. DEVICE_LOCAL describes location, not mappability. A type can be both device-local and host-visible. Heap size is not currently available budget.

Related sections: [14.5]

5.2 Requirements & suballocation

```
typedef struct PmMemoryRequirements {
    PmDeviceSize size;
    PmDeviceSize alignment;
    uint32_t memoryTypeBits;
    PmBool32 requiresDedicatedAllocation;
} PmMemoryRequirements;
```

Choose a type t for which memoryTypeBits & (1u << t) is nonzero. Bind at an offset divisible by alignment. The allocation must contain the full required range. Dedicated resources occupy a whole allocation at offset zero.

Suballocate distinct, non-overlapping ranges from larger blocks. Live bindings must not overlap, including scratch. Free a block only after destroying all bound resources and observing GPU completion. Requirements include resource-internal GPU data; bindings are immutable until destruction.

Related sections: [6.2], [7.2]

5.3 Allocate & free

```
typedef struct PmMemoryAllocateInfo {
    PmStructureType sType;
    const void* pNext;
    PmDeviceSize allocationSize;
    uint32_t memoryTypeIndex;
} PmMemoryAllocateInfo;

PmResult pmAllocateMemory(PmDevice device,
    const PmMemoryAllocateInfo* pInfo,
    PmDeviceMemory* pMemory);
void pmFreeMemory(PmDevice device,
    PmDeviceMemory memory);
```

allocationSize is nonzero. Allocation failure is explicit; no fallback silently changes the memory type. Pipeline code storage and driver bookkeeping are implementation-owned and reported separately; simulation, geometry acceleration and buffer storage are application-bound.

5.4 Map & host visibility

```
typedef struct PmMappedMemoryRange {
    PmDeviceMemory memory;
    PmDeviceSize offset;
    PmDeviceSize size;
} PmMappedMemoryRange;

PmResult pmMapMemory(PmDevice device,
    PmDeviceMemory memory,
    PmDeviceSize offset, PmDeviceSize size,
    void** ppData);
void pmUnmapMemory(PmDevice device,
    PmDeviceMemory memory);
PmResult pmFlushMappedMemoryRanges(
    PmDevice device, uint32_t count,
    const PmMappedMemoryRange* pRanges);
PmResult pmInvalidateMappedMemoryRanges(
    PmDevice device, uint32_t count,
    const PmMappedMemoryRange* pRanges);
```

Map HOST_VISIBLE memory once per allocation; the returned pointer addresses offset. PM_WHOLE_SIZE extends to allocation end. Flush host writes before GPU consumption. Wait for GPU writes before invalidating and reading. Noncoherent ranges start on atomSize boundaries and end on a boundary or allocation end; they must lie in the mapped range. Unmap does not flush. COHERENT removes cache-maintenance calls, not race-prevention duties.

Related sections: [15.1], [12.2]

6 Buffers & triangle geometry

One rigid geometry model • explicit storage and build work

6.1 Buffer creation

```
enum {
    PM_BUFFER_TRANSFER_SRC_BIT = 0x1,
    PM_BUFFER_TRANSFER_DST_BIT = 0x2,
    PM_BUFFER_GEOMETRY_INPUT_BIT = 0x4,
    PM_BUFFER_PARTICLE_INPUT_BIT = 0x8,
    PM_BUFFER_SIMULATION_OUTPUT_BIT = 0x10
};
typedef struct PmBufferCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmDeviceSize size;
    PmFlags usage;
    uint32_t initialQueueFamilyIndex;
} PmBufferCreateInfo;

PmResult pmCreateBuffer(PmDevice device,
    const PmBufferCreateInfo* pInfo,
    PmBuffer* pBuffer);
void pmDestroyBuffer(PmDevice device,
    PmBuffer buffer);
```

Creation declares a resource; it does not allocate its storage. All buffers have exclusive queue-family ownership. size is nonzero. Declare every usage required for the resource's lifetime.

6.2 Buffer requirements & binding

```
typedef struct PmBufferRange {
    PmBuffer buffer;
    PmDeviceSize offset;
    PmDeviceSize size;
} PmBufferRange;

void pmGetBufferMemoryRequirements(
    PmDevice device, PmBuffer buffer,
    PmMemoryRequirements* pRequirements);
PmResult pmBindBufferMemory(PmDevice device,
    PmBuffer buffer, PmDeviceMemory memory,
    PmDeviceSize offset);
```

Bind exactly once before use. Buffer ranges are bounds-checked, nonempty, and measured in bytes; PM_WHOLE_SIZE is allowed only for APIs that explicitly say so. Imported Vulkan buffers are already bound and must not be rebound.

Related sections: [5.2], [13]

6.3 Element formats

```
typedef enum PmFormat {
    PM_FORMAT_UNDEFINED = 0,
    PM_FORMAT_R32_UINT = 1,
    PM_FORMAT_R32_SFLOAT = 2,
    PM_FORMAT_R32G32B32_SFLOAT = 3,
    PM_FORMAT_R32G32B32A32_SFLOAT = 4,
    PM_FORMAT_PARTICLE_STATE = 5,
    PM_FORMAT_RIGID_STATE = 6
} PmFormat;
typedef struct PmBufferView {
    PmBufferRange range;
    PmFormat format;
    uint32_t count;
    uint32_t stride;
} PmBufferView;
```

A format defines byte interpretation, not a collider or solver type. stride is in bytes, at least the element size and a multiple of its alignment. count is nonzero; the final element must fit in range.size. Query usage support before choosing a format.

Related sections: [15.2]

6.4 Triangle geometry

```
typedef struct PmGeometryCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmBufferView vertices;
    PmBufferView indices;
    uint32_t queueFamilyIndex;
} PmGeometryCreateInfo;

PmResult pmCreateGeometry(PmDevice device,
    const PmGeometryCreateInfo* pInfo,
    PmGeometry* pGeometry);
void pmDestroyGeometry(PmDevice device,
    PmGeometry geometry);
void pmGetGeometryMemoryRequirements(
    PmDevice device, PmGeometry geometry,
    PmMemoryRequirements* pRequirements);
PmResult pmBindGeometryMemory(
    PmDevice device, PmGeometry geometry,
    PmDeviceMemory memory,
    PmDeviceSize offset);
```

Vertices use R32G32B32_SFLOAT; indices use R32_UINT and a count divisible by three. Indices must be in bounds; coordinates finite; triangles nondegenerate. Dynamic bodies require closed, convex, consistently wound meshes. Nonconvex static geometry is outside the core specification. Spheres and boxes are authored as triangle meshes.

Related sections: [7.4]

6.5 Recorded geometry build

```
PmResult pmCmdBuildGeometry(
    PmCommandBuffer commandBuffer,
    PmGeometry geometry);
```

Build after uploading and making input writes visible. Build creates acceleration data in bound storage and includes its internal dependencies. Barrier GEOMETRY_BUILD/WRITE → PHYSICS/READ before the first solve. Rebuild only when no dependent simulation is using the geometry. Inputs and their bindings remain alive for the entire geometry lifetime, not just the build. One geometry can be instanced by many bodies.

Related sections: [12.2], [9.5]

6.6 Resource creation checklist

1 · Describe	Size, usage, format, capacity, owner family
2 · Query	Exact size, alignment and type bits
3 · Allocate	Choose a compatible memory type
4 · Bind	Reserve a non-overlapping aligned range
5 · Initialize	Record upload / reset / build
6 · Depend	Make producer writes visible to users
7 · Retire	Complete GPU work, destroy, then free

7 Simulation state & rigid bodies

Mutable state lives separately from immutable solver configuration

7.1 Capacity-based simulation

```
typedef struct PmFluidCapacity {
    uint32_t particleCapacity;
    uint32_t neighborCapacityPerParticle;
} PmFluidCapacity;
typedef struct PmSimulationCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t queueFamilyIndex;
    uint32_t rigidBodyCapacity;
    uint32_t constraintCapacity;
    uint32_t contactCapacity;
    uint32_t fluidCount;
    const PmFluidCapacity* pFluidCapacities;
} PmSimulationCreateInfo;

PmResult pmCreateSimulation(PmDevice device,
    const PmSimulationCreateInfo* pInfo,
    PmSimulation* pSimulation);
void pmDestroySimulation(PmDevice device,
    PmSimulation simulation);
```

At least one body slot or fluid is required. Zero capacity disables a domain. Body and constraint IDs are stable, zero-based slots, not pointers. Each fluid is a separate particle state range with its own material. No hidden growth is permitted; change capacity by creating and migrating to a new simulation.

7.2 Persistent & scratch bindings

```
typedef struct PmSimulationMemoryRequirements {
    PmMemoryRequirements persistent;
    PmMemoryRequirements scratch;
} PmSimulationMemoryRequirements;
typedef struct PmSimulationMemoryBinding {
    PmDeviceMemory persistentMemory;
    PmDeviceSize persistentOffset;
    PmDeviceMemory scratchMemory;
    PmDeviceSize scratchOffset;
} PmSimulationMemoryBinding;

void pmGetSimulationMemoryRequirements(
    PmDevice device, PmSimulation simulation,
    PmSimulationMemoryRequirements* pInfo);
PmResult pmBindSimulationMemory(
    PmDevice device, PmSimulation simulation,
    const PmSimulationMemoryBinding* pBinding);
```

Requirements depend on declared capacities and include bounded contact/neighbor workspaces. Bind before reset or execution. Persistent and scratch ranges cannot overlap. Neither is a renderer-readable ABI. Use explicit output commands for portable state. Simulation state is exclusive to its creation queue family in core 0.2; share through output buffers.

Related sections: [5.3], [11.2]

7.3 Portable rigid state record

```
typedef struct PmRigidState {
    float position[3];
    float reserved0;
    float orientation[4];
    float linearVelocity[3];
    float reserved1;
    float angularVelocity[3];
    float reserved2;
} PmRigidState;
typedef enum PmMotionType {
    PM_MOTION_STATIC = 0,
    PM_MOTION_KINEMATIC = 1,
    PM_MOTION_DYNAMIC = 2
} PmMotionType;
```

The portable buffer record is 64 bytes with four 16-byte lanes. Reserved floats are zero. Quaternion identity is (0,0,0,1). Angular velocity is radians/second in world space. Static bodies do not integrate; kinematic states are application-driven and do not respond to forces.

Related sections: [15.2]

7.4 Body descriptions

```
typedef struct PmRigidBodyInfo {
    PmBool32 active;
    PmMotionType motionType;
    PmGeometry geometry;
    PmRigidState state;
    float mass;
    float inertiaDiagonal[3];
    float friction;
    float restitution;
    float linearDamping;
    float angularDamping;
    float collisionMargin;
} PmRigidBodyInfo;

PmResult pmCmdWriteRigidBodies(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmRigidBodyInfo* pBodies);
```

Host descriptions are copied during recording, including state; handles remain borrowed. Writing replaces each addressed slot. Dynamic bodies require positive mass and principal inertia about their center of mass, in kg·m². No implicit inertia approximation. Restitution is [0,1]; friction, damping and margin are nonnegative. Inactive entries require only active = PM_FALSE; all other fields are ignored. Remove or disable referencing constraints first.

7.5 Forces, impulses & targets

```
typedef struct PmRigidInput {
    uint32_t bodyIndex;
    float force[3];
    float torque[3];
    float linearImpulse[3];
    float angularImpulse[3];
} PmRigidInput;

PmResult pmCmdApplyRigidInputs(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t count, const PmRigidInput* pInputs);
PmResult pmCmdSetRigidStates(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmRigidState* pStates);
```

Inputs accumulate for the next step, then clear. Forces apply across that step's substeps; impulses apply once, not once per substep. Units: N, N·m, N·s, N·m·s. Dynamic state edits and inputs wake bodies. SetRigidStates replaces pose and velocity without changing material/geometry; use it for kinematic targets or explicit pose changes.

8 Rigid constraints

Contacts and joints enter the same AVBD solve

8.1 Joint families

```
typedef enum PmConstraintType {
    PM_CONSTRAINT_FIXED = 0,
    PM_CONSTRAINT_POINT = 1,
    PM_CONSTRAINT_HINGE = 2,
    PM_CONSTRAINT_SLIDER = 3,
    PM_CONSTRAINT_PISTON = 4,
    PM_CONSTRAINT_GENERIC = 5,
    PM_CONSTRAINT_GENERIC_SPRING = 6,
    PM_CONSTRAINT_MOTOR = 7
} PmConstraintType;
typedef struct PmConstraintFrame {
    float position[3];
    float orientation[4];
} PmConstraintFrame;
```

Frames are local to their body centers of mass. If bodyB is PM_INVALID_INDEX, frameB is in world space. Hinge, slider and piston axes are local +X of frameA. Both quaternions are normalized. A body cannot constrain itself.

Related sections: [9.1]

8.2 Per-axis controls

```
typedef struct PmAxisLimit {
    PmBool32 enabled;
    float lower;
    float upper;
} PmAxisLimit;
typedef struct PmAxisSpring {
    PmBool32 enabled;
    float stiffness;
    float damping;
    float equilibrium;
} PmAxisSpring;
typedef struct PmAxisMotor {
    PmBool32 enabled;
    float targetVelocity;
    float maximumForce;
} PmAxisMotor;
```

Linear limits/equilibria are meters; angular ones radians. lower ≤ upper; equality locks an axis. Disabled limits leave eligible axes free. Springs use nonnegative N/m and N·s/m, or rotational N·m/rad and N·m·s/rad. Motors use m/s and N, or rad/s and N·m. These are force/torque limits, not frame-dependent impulse limits.

8.3 Constraint description

```
typedef struct PmRigidConstraintInfo {
    PmBool32 active;
    PmConstraintType type;
    uint32_t bodyA;
    uint32_t bodyB;
    PmConstraintFrame frameA;
    PmConstraintFrame frameB;
    PmAxisLimit linearLimits[3];
    PmAxisLimit angularLimits[3];
    PmAxisSpring linearSprings[3];
    PmAxisSpring angularSprings[3];
    PmAxisMotor linearMotors[3];
    PmAxisMotor angularMotors[3];
    PmBool32 collideConnected;
} PmRigidConstraintInfo;

PmResult pmCmdWriteRigidConstraints(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstConstraint, uint32_t count,
    const PmRigidConstraintInfo* pConstraints);
```

Writing replaces stable slots. Inactive slots ignore other fields. Each active endpoint must refer to an active body or the world sentinel, and at least one endpoint must be dynamic. collideConnected = false suppresses that body's pair contact while the joint is active. No per-joint private solver or iteration loop exists.

8.4 Degrees of freedom

FIXED	All translation and rotation locked
POINT	Three translations locked; rotation free
HINGE	Only local X rotation eligible
SLIDER	Only local X translation eligible
PISTON	Local X translation and rotation eligible
GENERIC	All six axes eligible for limits/motors
GENERIC_SPRING	All six axes; limits, springs and motors
MOTOR	All axes free except driven/limited axes

Controls on intrinsically locked axes must be disabled. Springs are allowed only for GENERIC_SPRING; limit/motor controls apply on eligible axes for all other families. Incompatible combinations fail validation instead of being silently ignored.

8.5 Point joint to the world

A body-local anchor one meter above the body's origin is held at a world anchor three meters high. The body can rotate freely. Its geometry, body slot and simulation storage must already exist. Submit the write before the next solve with the required update-to-physics dependency.

Example (informative)

```
PmRigidConstraintInfo joint = {0};
joint.active = PM_TRUE;
joint.type = PM_CONSTRAINT_POINT;
joint.bodyA = 0;
joint.bodyB = PM_INVALID_INDEX;
joint.frameA.position[1] = 1.0f;
joint.frameA.orientation[3] = 1.0f;
joint.frameB.position[1] = 3.0f;
joint.frameB.orientation[3] = 1.0f;
PmResult result = pmCmdWriteRigidConstraints(
    cmd, sim, 0, 1, &joint);
```

Related sections: [12.2]

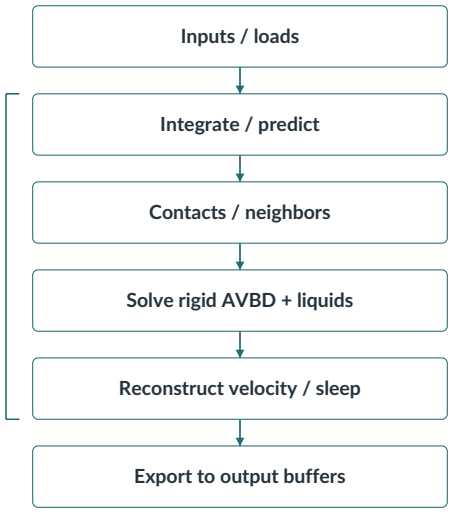
8.6 Iteration accounting

Pipeline rigidIterations is the per-phase budget shared by contacts and joints. The pose phase runs for each substep; impacted islands can also run a velocity-impact phase. Statistics report actual pose and impact iterations separately and sum executed work across substeps. One iteration is a valid budget, but does not guarantee convergence.

Related sections: [14.3], [9.1]

9 Physics pipelines & materials

Immutable configuration • explicit inter-domain coupling



Bracketed work repeats per substep.
Contacts + joints share the rigid solve.
Couplings connect declared domains.

9.1 Rigid solver state

```
typedef struct PmRigidSolverState {
    uint32_t rigidIterations;
    PmBool32 enableSleeping;
    float sleepLinearThreshold;
    float sleepAngularThreshold;
    float sleepTimeSeconds;
} PmRigidSolverState;
```

One AVBD path solves rigid geometry contacts and joints. rigidIterations is nonzero and ≤ maxRigidIterations. Sleeping thresholds are nonnegative m/s and rad/s; sleep time is positive when enabled.

9.2 Particle-liquid material

```
typedef struct PmLiquidMaterialState {
    float restDensity;
    float particleRadius;
    float supportRadius;
    float restParticleVolume;
    float pressureStiffness;
    float kinematicViscosity;
    float velocityDamping;
    float maximumSpeed;
    uint32_t densityIterations;
} PmLiquidMaterialState;
```

Density: kg/m^3; radii: m; rest volume: m^3; pressure stiffness: Pa; kinematic viscosity: m^2/s; damping: s^{-1}; speed: m/s. Density, radii, volume, maximumSpeed and iterations are positive; supportRadius ≥ 2 × particleRadius. Pressure stiffness, viscosity and damping are nonnegative. Material index equals the simulation's fluid index.

Materials configure a particle-liquid domain. Smoke, gas grids and non-Newtonian rheology are outside this material model.

9.3 Coupling state

```
typedef enum PmCouplingType {
    PM_COUPLING_LIQUID_RIGID = 0,
    PM_COUPLING_LIQUID_LIQUID = 1
} PmCouplingType;
typedef struct PmCouplingState {
    PmCouplingType type;
    uint32_t fluidA;
    uint32_t fluidB;
    float contactDistance;
    float normalStiffness;
    float normalDamping;
    float tangentialDrag;
    float maximumForce;
    PmBool32 twoWay;
} PmCouplingState;
```

LIQUID_RIGID connects fluidA to all active rigid bodies; fluidB must be PM_INVALID_INDEX. LIQUID_LIQUID requires distinct fluid indices and twoWay = true. No entry means no cross-domain interaction for that pair. Duplicate pairs are invalid.

Coefficients use m, N/m, N·s/m, N·s/m and N. All are nonnegative; maximumForce is positive. Two-way coupling applies equal-and-opposite pair impulses, before fixed/kinematic response constraints. One-way liquid/rigid coupling treats bodies as nonreactive obstacles. This is mechanical coupling, not automatic mixing, buoyancy, chemistry or heat exchange.

9.4 Pipeline creation

```
typedef struct PmPhysicsPipelineCreateInfo {
    PmStructureType sType;
    const void* pNext;
    const PmRigidSolverState* pRigidState;
    uint32_t fluidMaterialCount;
    const PmLiquidMaterialState* pFluidMaterials;
    uint32_t couplingCount;
    const PmCouplingState* pCouplings;
} PmPhysicsPipelineCreateInfo;

PmResult pmCreatePhysicsPipeline(
    PmDevice device,
    const PmPhysicsPipelineCreateInfo* pInfo,
    PmPhysicsPipeline* pPipeline);
void pmDestroyPhysicsPipeline(
    PmDevice device,
    PmPhysicsPipeline pipeline);
```

Creation copies all settings. pRigidState may be NULL only when no rigid domain is needed. The material count must equal a bound simulation's fluid count. Pipelines are immutable and shareable across compatible simulations. Changing materials creates a new pipeline, without reallocating state if its capacity requirements remain satisfied.

9.5 What a step records

pmCmdStepSimulation records a complete operation, not one CPU call per internal kernel. Its implementation owns dependencies among integration, broad phase, contact/neighbor generation, solve/coupling, velocity reconstruction and sleep. Applications supply dependencies between uploads, steps, exports, and other APIs.

A substep is a slice of physical time. An iteration refines constraints within that slice. Neither is a renderer frame. The API exposes the work budget explicitly and the queries report work actually executed.

Related sections: [10.5], [14.3]

9.6 Compatibility & bounds

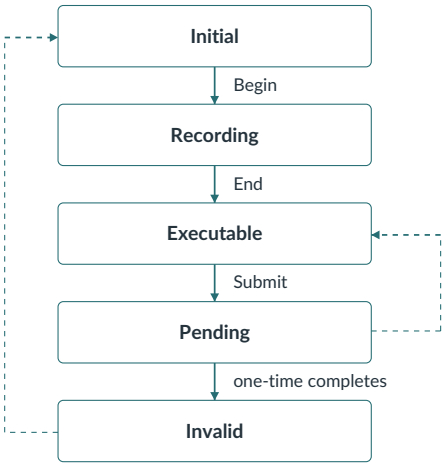
A rigid simulation requires pRigidState and enabled rigidBodies. Live constraints additionally require rigidConstraints. Each active fluid requires particleLiquids. Each coupling requires its matching enabled feature. A disabled domain cannot allocate hidden capacity.

All numerical inputs must be finite. Neighbors, contacts and output ranges are bounded. If runtime generation exceeds capacity, the step reports CAPACITY_EXCEEDED and its output is invalid; the application must reset/reinitialize before reuse. There is no silent truncation advertised as correct physics.

Related sections: [7.1], [14.3]

10 Command recording & lifecycle

Record now; execute when a queue receives the buffer



Reusable completion → Executable

Reset → Initial; never while Pending

10.1 Command pools

```
enum {
    PM_COMMAND_POOL_RESET_BUFFER_BIT = 0x1,
    PM_COMMAND_BUFFER_ONE_TIME_BIT = 0x1
};
typedef struct PmCommandPoolCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmFlags flags;
    uint32_t queueFamilyIndex;
} PmCommandPoolCreateInfo;
```

```
PmResult pmCreateCommandPool(PmDevice device,
    const PmCommandPoolCreateInfo* pInfo,
    PmCommandPool* pPool);
void pmDestroyCommandPool(PmDevice device,
    PmCommandPool pool);
PmResult pmResetCommandPool(PmDevice device,
    PmCommandPool pool);
```

A pool belongs to one family and is externally synchronized, including all recording operations on its buffers. Use one pool per recording thread. Reset returns every buffer to Initial; no buffer may be Pending. Destroying a pool frees its command buffers, after completion.

10.2 Allocate, record & reset

```
typedef struct PmCommandBufferBeginInfo {
    PmStructureType sType;
    const void* pNext;
    PmFlags flags;
} PmCommandBufferBeginInfo;

PmResult pmAllocateCommandBuffers(
    PmDevice device, PmCommandPool pool,
    uint32_t count, PmCommandBuffer* pBuffers);
void pmFreeCommandBuffers(PmDevice device,
    PmCommandPool pool, uint32_t count,
    const PmCommandBuffer* pBuffers);
PmResult pmBeginCommandBuffer(
    PmCommandBuffer commandBuffer,
    const PmCommandBufferBeginInfo* pInfo);
PmResult pmEndCommandBuffer(
    PmCommandBuffer commandBuffer);
PmResult pmResetCommandBuffer(
    PmCommandBuffer commandBuffer);
```

Primary buffers only. Begin requires Initial. End requires Recording and no latched error or open query. Individual reset requires RESET_BUFFER on the pool. No simultaneous submission of one command buffer. Host input arrays are copied at record time; no device work occurs while recording.

10.3 Legal states & resource lifetime

Initial	Begin → Recording; may free/reset
Recording	pmCmd* only; End → Executable
Executable	QueueSubmit → Pending; may reset/free
Pending	No edits, reset, free or resubmit
Completion	Reusable → Executable; one-time → Invalid
Invalid	Reset → Initial, or free

A recording failure makes End fail and leaves the buffer Invalid. Destroying a referenced nonpending resource invalidates dependent buffers. Destroying a pending resource is invalid usage. Application code must preserve pipeline, geometry, buffers, query pools and memory until completion. Pipeline bindings do not carry across buffers.

Related sections: [12.4], [12.5]

10.4 Reset & particle initialization

```
typedef struct PmParticleState {
    float position[3];
    float temperatureKelvin;
    float velocity[3];
    float reserved;
} PmParticleState;

PmResult pmCmdResetSimulation(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation);
PmResult pmCmdSetFluidParticles(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t fluidIndex,
    const PmBufferView* pParticles);
```

Reset is required after binding memory. It clears active bodies, joints, particles, loads, sleep history and step status; it preserves capacities. SetFluidParticles replaces one fluid population from PARTICLE_STATE input. count ≤ capacity; count zero clears the population and ignores range/format/stride. Nonzero input requires PARTICLE_INPUT usage. Temperature is carried state; the liquid model does not solve heat transfer.

10.5 Bind & step

```
typedef struct PmStepInfo {
    float timestep;
    uint32_t substeps;
    float gravity[3];
} PmStepInfo;

PmResult pmCmdBindPhysicsPipeline(
    PmCommandBuffer commandBuffer,
    PmPhysicsPipeline pipeline);
PmResult pmCmdStepSimulation(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    const PmStepInfo* pStep);
```

timestep > 0 seconds; 1 ≤ substeps ≤ maxSubsteps. Each substep advances timestep/substeps. Time advances only by the supplied timestep. A compatible pipeline must be bound. The simulation must be initialized, geometry built, and prior writes visible. Two submissions must not mutate one simulation concurrently.

Related sections: [9], [12.2]

11 Data movement & output

Renderer-visible buffers without exposing private solver layouts

11.1 Buffer transfers

```
typedef struct PmBufferCopy {
    PmDeviceSize srcOffset;
    PmDeviceSize dstOffset;
    PmDeviceSize size;
} PmBufferCopy;

PmResult pmCmdCopyBuffer(
    PmCommandBuffer commandBuffer,
    PmBuffer src, PmBuffer dst,
    uint32_t count,
    const PmBufferCopy* pRegions);
```

Use TRANSFER_SRC and TRANSFER_DST buffer usage. Regions must be in bounds, nonzero, four-byte aligned, and must not overlap through aliases. Host staging writes require a flush if noncoherent and must complete before submission. Insert a transfer-write dependency before a subsequent build or input read.

11.2 Copy simulation output

```
PmResult pmCmdCopyRigidStates(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmBufferRange* pDestination);
PmResult pmCmdCopyFluidParticles(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t fluidIndex,
    const PmBufferRange* pDestination);
```

Destination requires SIMULATION_OUTPUT and TRANSFER_DST usage. Rigid records are tightly packed 64-byte PmRigidState values in slot order; inactive slots produce zero position/velocities and identity orientation. Fluid records are tightly packed 32-byte PmParticleState values in population order. The destination must cover the addressed count, or the fluid's declared particle capacity. Bytes beyond active records are unchanged.

These commands execute in TRANSFER and read simulation state after a PHYSICS_WRITE → TRANSFER_READ dependency. A subsequent renderer needs a TRANSFER_WRITE dependency. Output copies make the portable contract independent of private SoA solver storage.

Related sections: [15.2], [12.2], [13.9]

11.3 Frame sequence (pseudocode)

Updates and state changes are recorded commands. The next frame needs a dependency on this frame's simulation reads/writes, not only on the renderer buffer. Reusing one output buffer requires waiting for its renderer consumer.

Example (informative)

```
begin(cmd);
  resetQueries();
  writeTimestamp(start);
  applyInputs(sim);
  barrier(UPDATE_WRITE, PHYSICS_READ_WRITE);
  bindPhysicsPipeline(pipeline);
  beginStatistics(stats);
  stepSimulation(sim, dt, substeps);
  endStatistics(stats);
  barrier(PHYSICS_WRITE, TRANSFER_READ);
  copyRigidStates(sim, renderBuffer);
  writeTimestamp(end);
end(cmd);
submit(waitRender, cmd, signalPhysics, fence);
```

Related sections: [16], [12.4]

11.4 Reusable lists & frames in flight

Recorded commands capture scalar settings and host records by value. Editing the original C struct does not change an executable command buffer. Re-record changing inputs or use GPU input buffers with explicit ownership and dependencies.

Reusable buffers can return to Executable after completion. Multiple frames in flight need separate command buffers and output slots, plus timeline dependencies that serialize mutation of one simulation. More command buffers do not make one physics state safely concurrent. Use distinct simulations for independent workloads.

11.5 Allocation and execution boundaries

The implementation may allocate CPU command-recording storage in a pool and GPU code/driver bookkeeping at pipeline creation. All physics persistent/scratch storage comes from the declared bindings. Record and submit must not grow physics device allocations. Geometry rebuild, reset, state uploads and result copies are visible commands; the application can put them on its own frame graph.

12 Submission & synchronization

Execution order, memory visibility and ownership are separate obligations

12.1 Stage and access scopes

```
#define PM_STAGE_HOST_BIT UINT64_C(0x1)
#define PM_STAGE_TRANSFER_BIT UINT64_C(0x2)
#define PM_STAGE_UPDATE_BIT UINT64_C(0x4)
#define PM_STAGE_GEOMETRY_BUILD_BIT UINT64_C(0x8)
#define PM_STAGE_PHYSICS_BIT UINT64_C(0x10)
#define PM_STAGE_ALL_COMMANDS_BIT UINT64_C(0x1e)
#define PM_ACCESS_HOST_READ_BIT UINT64_C(0x1)
#define PM_ACCESS_HOST_WRITE_BIT UINT64_C(0x2)
#define PM_ACCESS_READ_BIT UINT64_C(0x4)
#define PM_ACCESS_WRITE_BIT UINT64_C(0x8)
```

READ/WRITE are generic device accesses narrowed by the stage mask. UPDATE covers reset, body/constraint/state writes and particle initialization. TRANSFER covers buffer and output copies. ALL_COMMANDS covers device stages, not HOST. A host flush is not a GPU completion wait.

12.2 Memory & buffer barriers

```
typedef struct PmMemoryBarrier {
    PmStageFlags srcStages;
    PmAccessFlags srcAccess;
    PmStageFlags dstStages;
    PmAccessFlags dstAccess;
} PmMemoryBarrier;
typedef struct PmBufferMemoryBarrier {
    PmMemoryBarrier memory;
    PmBufferRange range;
    uint32_t srcQueueFamilyIndex;
    uint32_t dstQueueFamilyIndex;
} PmBufferMemoryBarrier;

PmResult pmCmdPipelineBarrier(
    PmCommandBuffer commandBuffer,
    uint32_t memoryBarrierCount,
    const PmMemoryBarrier* pMemoryBarriers,
    uint32_t bufferBarrierCount,
    const PmBufferMemoryBarrier* pBufferBarriers);
```

Memory barriers cover all matching device accesses, including opaque simulation storage. Buffer barriers narrow scope and can transfer ownership. Use IGNORED for both family indices when ownership is unchanged. For a family transfer, record a release on the source and a matching acquire on the destination, with a semaphore between queues. Release uses zero destination stage/access; acquire uses zero source stage/access. Matching ranges and family pairs are required.

Related sections: [13.9]

12.3 Timeline semaphores

```
typedef struct PmSemaphoreCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint64_t initialValue;
} PmSemaphoreCreateInfo;
typedef struct PmSemaphoreSubmitInfo {
    PmSemaphore semaphore;
    uint64_t value;
    PmStageFlags stages;
} PmSemaphoreSubmitInfo;

PmResult pmCreateSemaphore(PmDevice device,
    const PmSemaphoreCreateInfo* pInfo,
    PmSemaphore* pSemaphore);
void pmDestroySemaphore(PmDevice device,
```

```
    PmSemaphore semaphore);
PmResult pmGetSemaphoreCounterValue(
    PmDevice device, PmSemaphore semaphore,
    uint64_t* pValue);
PmResult pmWaitSemaphore(PmDevice device,
    PmSemaphore semaphore, uint64_t value,
    uint64_t timeoutNanoseconds);
```

Waits complete at counter ≥ value. Signal values increase strictly and fit within the device's maximum pending difference. Core signals occur through submission, not a host signal call. A timeline is not reset. Use separate timelines when producers cannot order their signal values.

12.4 Queue submission

```
typedef struct PmSubmitInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t waitCount;
    const PmSemaphoreSubmitInfo* pWaits;
    uint32_t commandBufferCount;
    const PmCommandBuffer* pCommandBuffers;
    uint32_t signalCount;
    const PmSemaphoreSubmitInfo* pSignals;
} PmSubmitInfo;

PmResult pmQueueSubmit(PmQueue queue,
    const PmSubmitInfo* pInfo, PmFence fence);
PmResult pmQueueWaitIdle(PmQueue queue);
PmResult pmDeviceWaitIdle(PmDevice device);
```

Submit accepts Executable buffers from the queue's family. A successful call means accepted, not complete. Wait scopes name the first consuming device stages; signal scopes must be ALL_COMMANDS. Signals publish prior writes; waits make them visible in the chosen stages. Ownership transfers may still be necessary. fence may be NULL. WaitIdle is useful for shutdown, not required between frames.

12.5 Host completion fences

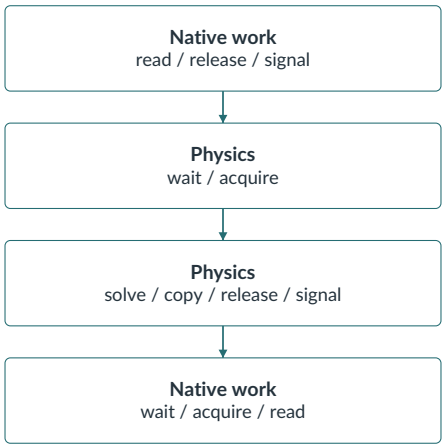
```
enum { PM_FENCE_SIGNALED_BIT = 0x1 };

PmResult pmCreateFence(PmDevice device,
    PmFlags flags, PmFence* pFence);
void pmDestroyFence(PmDevice device,
    PmFence fence);
PmResult pmGetFenceStatus(PmDevice device,
    PmFence fence);
PmResult pmResetFences(PmDevice device,
    uint32_t count, const PmFence* pFences);
PmResult pmWaitForFences(PmDevice device,
    uint32_t count, const PmFence* pFences,
    PmBool32 waitAll, uint64_t timeoutNanoseconds);
```

Submit requires an unsignaled fence not associated with pending work. Get returns SUCCESS or NOT_READY. Wait accepts zero for polling and PM_TIMEOUT_INFINITE for an unbounded wait; expiration returns TIMEOUT. Only reset a fence after its associated work completes. A fence synchronizes GPU completion with the CPU, not another GPU queue.

13 Native API interoperability

CUDA, Metal, DirectX 12 and Vulkan



Timeline signals connect GPU queues.

Native API sets visibility and resource state.

13.1 Backend capabilities

```
typedef enum PmNativeBackend {
    PM_NATIVE_BACKEND_CUDA = 1,
    PM_NATIVE_BACKEND_METAL = 2,
    PM_NATIVE_BACKEND_D3D12 = 3,
    PM_NATIVE_BACKEND_VULKAN = 4
} PmNativeBackend;
typedef uint64_t PmNativeHandle;
typedef struct PmNativeInteropProperties {
    PmBool32 deviceImport;
    PmBool32 bufferImport;
    PmBool32 timelineImport;
} PmNativeInteropProperties;

PmResult pmGetPhysicalDeviceInteropProperties(
    PmPhysicalDevice physicalDevice,
    PmNativeBackend backend,
    PmNativeInteropProperties* pProperties);
```

Query each native backend on the selected physical device before importing objects. An unsupported backend returns FEATURE_NOT_PRESENT and zero properties. Each import requires its own capability; support for one backend does not imply support for another. The vulkanInterop feature additionally gates the typed Vulkan extension entry points.

PmNativeHandle is an in-process carrier, not a file descriptor, Win32 shared handle, or serialized object. Encode native pointer handles through uintptr_t, then uint64_t; encode integer handles without changing their bits. Decode using the original native type. Zero means no object, except a CUDA physical-device ordinal may be zero. Imported native objects remain owned by the application. PM wrappers must be destroyed, and their work completed, before native objects are released.

Related sections: [4.4], [4.2]

13.2 Device and queue import

```
enum {
    PM_NATIVE_TIMELINE_ENABLED_BIT = 0x1,
    PM_NATIVE_SYNCHRONIZATION2_BIT = 0x2
};
typedef struct PmNativeQueueInfo {
    PmNativeHandle queue;
    uint32_t familyIndex;
    uint32_t queueIndex;
} PmNativeQueueInfo;
typedef struct PmNativeDeviceInfo {
    PmNativeBackend backend;
    PmNativeHandle instance;
    PmNativeHandle physicalDevice;
    PmNativeHandle device;
} PmNativeDeviceInfo;
typedef struct PmNativeDeviceImportInfo {
    PmStructureType sType;
    const void* pNext;
    PmNativeDeviceInfo nativeDevice;
    uint32_t queueCount;
    const PmNativeQueueInfo* pQueues;
    PmFlags enabledNativeFeatures;
    PmPhysicalDeviceFeatures enabledFeatures;
} PmNativeDeviceImportInfo;

PmResult pmImportNativeDevice(
    PmInstance instance,
    const PmNativeDeviceImportInfo* pInfo,
    PmDevice* pDevice);
PmResult pmGetNativeDevice(PmDevice device,
    PmNativeDeviceInfo* pNativeDevice);
```

Import borrows a device/context and at least one explicit queue/stream. enabledFeatures selects PM features; enabledNativeFeatures records native capabilities already enabled by the caller, not requests to enable them. Unknown bits fail. TIMELINE_ENABLED is required for shared counter synchronization. SYNCHRONIZATION2 is required only for Vulkan 1.3 import, and must be zero for other backends. Get returns borrowed handles, including for PM-created native devices; it does not transfer ownership.

CUDA uses instance = 0, physicalDevice = CUdevice ordinal, device = CUcontext, queue = a non-default CUstream from that context. Metal uses instance = physicalDevice = 0, device = id<MTLDevice>, queue = id<MTLCommandQueue>. DirectX 12 uses instance = physicalDevice = 0, device = ID3D12Device*, queue = ID3D12CommandQueue* of DIRECT or COMPUTE type. Vulkan uses VkInstance, VkPhysicalDevice, VkDevice and VkQueue. Verify adapter identity before combining native objects.

Vulkan retains native queue family/index values. Other backends use familyIndex = 0 and contiguous queueIndex values starting at zero; PM exposes these as its imported family. Native queues must belong to the supplied device. Serialize host-side access when both APIs submit through the same native queue. Imports do not adopt a caller's partially recorded command list or encoder.

Related sections: [12.4], [4.3]

13.3 Buffer import

```
typedef struct PmNativeBufferInfo {
    PmNativeHandle buffer;
    PmDeviceSize offset;
    PmDeviceSize size;
    PmFlags usage;
    uint64_t nativeUsage;
    uint32_t owningQueueFamilyIndex;
} PmNativeBufferInfo;

PmResult pmImportNativeBuffer(PmDevice device,
    const PmNativeBufferInfo* pInfo,
    PmBuffer* pBuffer);
```

```
PmResult pmGetNativeBuffer(PmDevice device,
    PmBuffer buffer,
    PmNativeBufferInfo* pNativeBuffer);
```

The imported buffer is already allocated and bound. size is nonzero; offset + size must fit the native allocation without overflow and satisfy PM alignment requirements. All PM offsets are relative to this imported range. No pmBindBufferMemory call is permitted on the wrapper. usage declares PM buffer usage and must be compatible with native storage. Get reports the same borrowed native object and range.

CUDA buffer is a CUdeviceptr, not a host pointer; its context must match. Metal buffer is id<MTLBuffer>; device and storage mode must permit GPU access. DirectX 12 buffer is an ID3D12Resource* with BUFFER dimension and appropriate resource flags; it enters and leaves PM ownership in D3D12_RESOURCE_STATE_COMMON. PM performs its internal state transitions. Set nativeUsage to D3D12_RESOURCE_FLAGS for DirectX 12 and to VkBufferUsageFlags for Vulkan; set it to zero for CUDA and Metal. Vulkan buffers use exclusive sharing and require usages matching the typed import contract.

owningQueueFamilyIndex is a Vulkan family for Vulkan; other backends use zero for PM ownership or PM_QUEUE_FAMILY_EXTERNAL for native ownership. On those backends, a PM buffer barrier with EXTERNAL → 0 acquires native storage; 0 → EXTERNAL releases it. The application brackets native access with the matching timeline wait/signal. Queue execution ordering does not remove resource-state or memory-visibility requirements.

Related sections: [12.2], [13.7]

13.4 Timeline import

```
PmResult pmImportNativeSemaphore(PmDevice device,
    PmNativeHandle nativeSemaphore,
    PmSemaphore* pSemaphore);
PmResult pmGetNativeSemaphore(PmDevice device,
    PmSemaphore semaphore,
    PmNativeHandle* pNativeSemaphore);
```

Import borrows the native counter object for the device backend. CUDA accepts CUexternalSemaphore imported as a timeline semaphore or D3D12 fence, not a CUevent or binary semaphore. Metal accepts id<MTLSharedEvent>; DirectX 12 accepts ID3D12Fence*; Vulkan accepts a timeline VkSemaphore. The object must be compatible with the native device/context and its counter values must obey PM's increasing-signal rule. Get returns the borrowed object; if no compatible native representation exists it clears the output and returns FEATURE_NOT_PRESENT. CUDA external-handle import and destruction remain the application's responsibility.

Related sections: [12.3]

13.5 Native object mapping

CUDA	CUcontext / CUstream; CUdeviceptr; CUexternalSemaphore
Metal	MTLDevice / MTLCommandQueue; MTLBuffer; MTLSharedEvent
DirectX 12	ID3D12Device / CommandQueue; Resource; Fence
Vulkan	VkDevice / VkQueue; VkBuffer; timeline VkSemaphore

13.6 Typed Vulkan device adapter

```
typedef struct PmVulkanQueueInfo {
    VkQueue queue;
    uint32_t familyIndex;
    uint32_t queueIndex;
} PmVulkanQueueInfo;
typedef struct PmVulkanDeviceInfo {
    VkInstance instance;
    VkPhysicalDevice physicalDevice;
    VkDevice device;
} PmVulkanDeviceInfo;
typedef struct PmVulkanDeviceImportInfo {
    VkInstance instance;
    VkPhysicalDevice physicalDevice;
    VkDevice device;
    uint32_t queueCount;
    const PmVulkanQueueInfo* pQueues;
    VkBool32 timelineSemaphoreEnabled;
    VkBool32 synchronization2Enabled;
    PmPhysicalDeviceFeatures enabledFeatures;
} PmVulkanDeviceImportInfo;

PmResult pmImportVulkanDeviceEXT(
    PmInstance instance,
    const PmVulkanDeviceImportInfo* pInfo,
    PmDevice* pDevice);
PmResult pmGetVulkanDeviceEXT(PmDevice device,
    PmVulkanDeviceInfo* pNativeDevice);
```

Declared in parallel_mater_vulkan.h. Requires Vulkan 1.3, timelineSemaphore and synchronization2 actually enabled at native device creation. Caller must report enabled state truthfully. Match an enumerated physical-device UUID with vulkanInterop support. Imports borrow every native object; PM destroys only its wrappers and internal children. Get returns borrowed handles for either an imported or PM-owned native device; never destroy PM-owned handles through Vulkan. Queue family/index values retain their Vulkan meanings.

13.7 Typed Vulkan buffer adapter

```
typedef struct PmVulkanBufferImportInfo {
    VkBuffer buffer;
    VkDeviceSize size;
    VkBufferUsageFlags nativeUsage;
    PmFlags usage;
    uint32_t owningQueueFamilyIndex;
} PmVulkanBufferImportInfo;

PmResult pmImportVulkanBufferEXT(
    PmDevice device,
    const PmVulkanBufferImportInfo* pInfo,
    PmBuffer* pBuffer);
PmResult pmGetVulkanBufferEXT(PmDevice device,
    PmBuffer buffer, VkBuffer* pNativeBuffer);
```

The native buffer must be bound, large enough, exclusive-sharing, on the same VkDevice and created with all declared native usages. PM input buffers need STORAGE_BUFFER; transfer source/destination usages must match. SIMULATION_OUTPUT needs TRANSFER_DST. Native renderer usages such as VERTEX_BUFFER remain the renderer's responsibility. Destroying an imported PM wrapper does not destroy its VkBuffer or memory.

13.8 Typed Vulkan timeline adapter

```
PmResult pmImportVulkanSemaphoreEXT(
    PmDevice device, VkSemaphore semaphore,
    PmSemaphore* pSemaphore);
PmResult pmGetVulkanSemaphoreEXT(
    PmDevice device, PmSemaphore semaphore,
    VkSemaphore* pNativeSemaphore);
```

Import borrows an existing timeline semaphore from the same device. Binary semaphores are invalid. Get returns a borrowed native handle; do not destroy it through Vulkan if PM owns it. The same increasing counter values can be used in PmSubmitInfo and VkSemaphoreSubmitInfo. No CPU wait is needed merely to connect the two queues.

13.9 Native work → physics → native work

Before overwriting a shared output slot, physics waits for the native consumer's done value, acquires the buffer, solves, copies state, and releases the buffer. It signals completion only after the writes and release. The native consumer waits, establishes its required visibility and resource state, reads the data, then releases it and signals a later done value. The same pattern applies to rendering, compute, or another physics producer.

CUDA enqueues external-semaphore waits and signals on streams. Metal encodes shared-event waits and signals on command buffers. DirectX 12 uses command-queue Wait/Signal with a fence and the appropriate resource transitions. Vulkan uses timeline semaphore waits/signals plus matching queue-family release/acquire where necessary. A shared queue does not eliminate memory hazards; host calls sharing that queue must be serialized. No CPU idle wait is required merely to connect supported native queues.

Related sections: [12.2], [12.4], [13.4]

13.10 Native synchronization

CUDA	Stream order + external timeline waits/signals
Metal	Command buffers + shared events + resource hazards
DirectX 12	Queue fences + resource barriers
Vulkan	Timeline semaphores + memory/ownership barriers

PM stages describe logical dependencies, not a native shader ABI. CUDA and Metal may conservatively synchronize complete earlier operations where a finer PM stage is unavailable. DirectX 12 must also transition shared resources at the ownership boundary. For Vulkan, TRANSFER maps to ALL_TRANSFER, UPDATE to COMPUTE_SHADER | ALL_TRANSFER, GEOMETRY_BUILD and PHYSICS to COMPUTE_SHADER, and ALL_COMMANDS to ALL_COMMANDS; PM read/write accesses map to MEMORY_READ/MEMORY_WRITE. The native consumer selects its actual consuming stage and access.

13.11 Sharing boundary

Importing an existing in-process native object is distinct from importing operating-system external-memory handles or sharing across processes. Allocate and import external resources through the native APIs first; pass the resulting compatible device-local objects to PM only when the relevant capability is reported. This contract does not promise arbitrary cross-API or cross-device sharing, handle export, or a host-copy fallback. Unsupported imports return FEATURE_NOT_PRESENT. The typed Vulkan adapters have the same ownership and capability requirements as the backend-neutral calls.

14 Timing, work & memory queries

Measure GPU execution separately from host overhead

14.1 Query pools

```
typedef enum PmQueryType {
    PM_QUERY_TIMESTAMP = 0,
    PM_QUERY_STEP_STATISTICS = 1
} PmQueryType;
typedef struct PmQueryPoolCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmQueryType queryType;
    uint32_t queryCount;
} PmQueryPoolCreateInfo;

PmResult pmCreateQueryPool(PmDevice device,
    const PmQueryPoolCreateInfo* pInfo,
    PmQueryPool* pPool);
void pmDestroyQueryPool(PmDevice device,
    PmQueryPool pool);
PmResult pmCmdResetQueryPool(
    PmCommandBuffer commandBuffer,
    PmQueryPool pool, uint32_t first,
    uint32_t count);
```

Query pools own implementation-managed result storage, accounted as library bytes. Reset slots before first use and after consuming prior results. Do not reset results still in flight. timestampQueries/statisticsQueries must be enabled for the selected type. queryCount is positive and bounded by maxQueriesPerPool.

14.2 GPU timestamps

```
PmResult pmCmdWriteTimestamp(
    PmCommandBuffer commandBuffer,
    PmStageFlags stage,
    PmQueryPool pool, uint32_t query);
```

Use ALL_COMMANDS for a portable before/after measurement. A timestamp becomes available once earlier commands have completed and the sample has been written. Timestamp values are unsigned 64-bit ticks; only timestampValidBits are meaningful. GPU nanoseconds = modular tick delta × timestampPeriodNanoseconds. Samples must be on the same queue, and duration must be less than one wrap period. They do not measure CPU recording, queue-submit overhead or wall-clock time.

14.3 Step statistics & execution status

```
typedef struct PmStepStatistics {
    uint64_t stepCount;
    uint64_t substepsExecuted;
    uint64_t poseIterations;
    uint64_t impactIterations;
    uint64_t peakContactCount;
    uint64_t peakActiveBodies;
    uint64_t peakAwakeBodies;
    uint64_t peakParticleCount;
    uint64_t scratchHighWaterBytes;
    uint64_t overflowCount;
    int32_t executionResult;
    uint32_t reserved;
} PmStepStatistics;

PmResult pmCmdBeginStatisticsQuery(
    PmCommandBuffer commandBuffer,
    PmQueryPool pool, uint32_t query);
PmResult pmCmdEndStatisticsQuery(
    PmCommandBuffer commandBuffer,
    PmQueryPool pool, uint32_t query);
PmResult pmGetSimulationStatus(
    PmDevice device, PmSimulation simulation,
    PmResult* pExecutionResult);
```

Queries accumulate across enclosed steps: work counters sum, peak fields and high-water bytes take maxima. executionResult is the first failing PmResult, or SUCCESS. Do not nest statistics queries; Begin and End must be in one buffer. After observing completion, GetSimulationStatus exposes the last step status even when statistics are disabled. It returns NOT_READY while simulation work is pending. Overflow invalidates that step's output and requires reinitialization.

14.4 Readback & availability

```
enum {
    PM_QUERY_RESULT_WAIT_BIT = 0x1,
    PM_QUERY_RESULT_WITH_AVAILABILITY_BIT = 0x2
};

PmResult pmGetQueryPoolResults(PmDevice device,
    PmQueryPool pool, uint32_t first,
    uint32_t count, size_t dataSize,
    void* pData, PmDeviceSize stride,
    PmFlags flags);
```

A timestamp result is uint64_t; statistics use PmStepStatistics (88 bytes). WITH_AVAILABILITY appends uint64_t, zero/unavailable or one/available, after each result. stride is an eight-byte multiple large enough for a record; pData is eight-byte aligned; dataSize covers all records. Without WAIT, unavailable result bytes remain untouched and NOT_READY is returned; availability fields are still written. WAIT has no timeout: use a fence or semaphore wait first for bounded waiting.

14.5 Device budget & library accounting

```
typedef struct PmMemoryBudget {
    uint32_t heapCount;
    PmDeviceSize budget[16];
    PmDeviceSize usage[16];
    PmDeviceSize applicationAllocationBytes;
    PmDeviceSize libraryDeviceBytes;
    PmDeviceSize libraryHostBytes;
    PmDeviceSize borrowedBufferBytes;
} PmMemoryBudget;

PmResult pmGetDeviceMemoryBudget(
    PmDevice device, PmMemoryBudget* pBudget);
```

Requires enabled memoryBudget. Budget/usage are live driver estimates per heap, not guaranteed free space or a reservation. Application bytes sum owned pmAllocateMemory blocks. Library bytes cover queries, pipeline code and internal bookkeeping. Borrowed bytes sum imported buffer sizes, not physical residency or unique allocations. These accounting categories are not additive with heap usage. Resource requirements describe reservation; scratchHighWaterBytes describes use.

Related sections: [5.1]

15 Features, limits & formats

Capability decisions are made before allocation and pipeline creation

15.1 Physical-device limits

```
typedef struct PmPhysicalDeviceLimits {
    PmDeviceSize maxAllocationSize;
    PmDeviceSize maxBufferSize;
    PmDeviceSize nonCoherentAtomSize;
    uint32_t maxMemoryAllocations;
    uint32_t maxRigidBodies;
    uint32_t maxRigidConstraints;
    uint32_t maxContacts;
    uint32_t maxFluidSystems;
    uint32_t maxParticlesPerFluid;
    uint32_t maxNeighborsPerParticle;
    uint32_t maxRigidIterations;
    uint32_t maxLiquidIterations;
    uint32_t maxSubsteps;
    uint32_t maxQueriesPerPool;
    uint64_t maxTimelineValueDifference;
} PmPhysicalDeviceLimits;

void pmGetPhysicalDeviceLimits(
    PmPhysicalDevice physicalDevice,
    PmPhysicalDeviceLimits* pLimits);
```

Limits are maxima for one resource or field, not performance targets. Memory pressure can still make a legal allocation fail. Every count and numerical budget must be checked against the corresponding limit before creation/recording.

15.2 Format support query

```
enum {
    PM_FORMAT_GEOMETRY_VERTEX_BIT = 0x1,
    PM_FORMAT_GEOMETRY_INDEX_BIT = 0x2,
    PM_FORMAT_PARTICLE_INPUT_BIT = 0x4,
    PM_FORMAT_SIMULATION_OUTPUT_BIT = 0x8
};
typedef struct PmFormatProperties {
    uint32_t elementSize;
    uint32_t elementAlignment;
    PmFlags supportedUsages;
} PmFormatProperties;

void pmGetPhysicalDeviceFormatProperties(
    PmPhysicalDevice physicalDevice,
    PmFormat format,
    PmFormatProperties* pProperties);
```

UNDEFINED or an unsupported format returns zero properties. Supported formats have the fixed layouts below; implementation-specific packed formats are not inferred. Query flags express usage support, not a separate simulation model.

15.3 Portable buffer layouts

R32_UINT	4 bytes / alignment 4 / unsigned index
R32_SFLOAT	4 bytes / alignment 4 / IEEE binary32
R32G32B32_SFLOAT	12 bytes / alignment 4 / x,y,z
R32G32B32A32_SFLOAT	16 bytes / alignment 4 / x,y,z,w
PARTICLE_STATE	32 bytes / alignment 4 / PmParticleState
RIGID_STATE	64 bytes / alignment 4 / PmRigidState

Record arrays are little-endian. These buffer layouts have fixed offsets, regardless of a shader language's native vector packing. PmRigidBodyInfo and other handle-containing host descriptors are not GPU binary layouts. Do not memcpy them into storage buffers. Scalar fields are 32-bit unless explicitly declared otherwise.

15.4 Dependency checklist

Rigid contacts	rigidBodies
Rigid joints	rigidBodies + rigidConstraints
One liquid	particleLiquids
Liquid ↔ bodies	both domains + liquidRigidCoupling
Liquid ↔ liquid	particleLiquids + liquidLiquidCoupling
GPU timing	timestampQueries + queue timestamp bits
Solver counters	statisticsQueries
Native object sharing	Per-backend import capabilities
Typed Vulkan adapters	vulkanInterop + required native features

Report a feature only when its required behavior is supported. Requesting unsupported features or coupling pairs fails with FEATURE_NOT_PRESENT.

Related sections: [4.4], [13.1]

15.5 Threading & data races

Physical-device queries are safe to call concurrently. Distinct command pools can record independently. Host access to the same queue, pool, fence, mapped range or mutable resource must be externally synchronized. Destroying an object must also be synchronized against queries that use it.

GPU races are a separate concern. Serialize writes to a simulation, coordinate shared input/output buffers, and avoid changing CPU-mapped bytes while a pending command may read them. Callback implementations must support concurrency across independently accessed objects.

15.6 Core scope

Core 0.2 defines primary command buffers, explicit memory binding, rigid triangle geometry, rigid constraints, particle liquids and the listed couplings. Secondary buffers, simultaneous reuse, sparse resources, shader injection and cross-device migration are outside this specification.

Cloth, ropes, soft bodies, smoke, emitters and paint require domain-specific extensions. Unknown pNext structures are errors.

Related sections: [4.4]

16 A frame, recorded explicitly

Record, submit and consume a simulation frame

16.1 Record a rigid-body frame

TRY returns the first non-success result. On any recording error, reset the buffer before trying again. This example requires initialized resources, completed prior command/query use, and a renderer output buffer in the same queue family. All referenced objects and memory must remain alive until completion.

Example (informative)

```
PmResult record_frame(PmCommandBuffer cmd,
    PmPhysicsPipeline pipeline, PmSimulation sim,
    PmBuffer output, uint32_t bodyCount,
    PmQueryPool timestamps, const PmStepInfo* step)
{
    PmCommandBufferBeginInfo begin;
    memset(&begin, 0, sizeof(begin));
    begin.sType = PM_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
    begin.flags = PM_COMMAND_BUFFER_ONE_TIME_BIT;
    TRY(pmResetCommandBuffer(cmd));
    TRY(pmBeginCommandBuffer(cmd, &begin));
    TRY(pmCmdResetQueryPool(cmd, timestamps, 0, 2));
    TRY(pmCmdWriteTimestamp(cmd,
        PM_STAGE_ALL_COMMANDS_BIT, timestamps, 0));

    PmMemoryBarrier before = {
        PM_STAGE_ALL_COMMANDS_BIT, PM_ACCESS_WRITE_BIT,
        PM_STAGE_PHYSICS_BIT,
        PM_ACCESS_READ_BIT | PM_ACCESS_WRITE_BIT
    };
    TRY(pmCmdPipelineBarrier(cmd, 1, &before, 0, NULL));
    TRY(pmCmdBindPhysicsPipeline(cmd, pipeline));
    TRY(pmCmdStepSimulation(cmd, sim, step));

    PmMemoryBarrier after = {
        PM_STAGE_PHYSICS_BIT, PM_ACCESS_WRITE_BIT,
        PM_STAGE_TRANSFER_BIT, PM_ACCESS_READ_BIT
    };
    TRY(pmCmdPipelineBarrier(cmd, 1, &after, 0, NULL));
    PmBufferRange destination = {
        output, 0, (PmDeviceSize)bodyCount * 64u
    };
    TRY(pmCmdCopyRigidStates(
        cmd, sim, 0, bodyCount, &destination));
    TRY(pmCmdWriteTimestamp(cmd,
        PM_STAGE_ALL_COMMANDS_BIT, timestamps, 1));
    return pmEndCommandBuffer(cmd);
}
```

Related sections: [10.3], [13.9]

16.2 Submit a frame between timeline values

The renderer signals renderDone after its previous read. Physics signals physicsDone after copying output. The renderer waits for that new physics value at its actual consuming stage. completion is an unsignaled fence, or NULL. Semaphore signal values must be globally monotonic.

Example (informative)

```
PmResult submit_frame(PmQueue queue, PmCommandBuffer cmd,
    PmSemaphore renderDone, uint64_t renderValue,
    PmSemaphore physicsDone, uint64_t physicsValue,
    PmFence completion)
{
    PmSemaphoreSubmitInfo wait = {
        renderDone, renderValue, PM_STAGE_ALL_COMMANDS_BIT
    };
    PmSemaphoreSubmitInfo signal = {
        physicsDone, physicsValue, PM_STAGE_ALL_COMMANDS_BIT
    };
    PmSubmitInfo submit;
    memset(&submit, 0, sizeof(submit));
    submit.sType = PM_STRUCTURE_TYPE_SUBMIT_INFO;
    submit.waitCount = 1;
    submit.pWaits = &wait;
    submit.commandBufferCount = 1;
    submit.pCommandBuffers = &cmd;
    submit.signalCount = 1;
    submit.pSignals = &signal;
    return pmQueueSubmit(queue, &submit, completion);
}
```

Related sections: [12.4]

16.3 Before the first frame

Instance / device	Enumerate; query support; enable features
Geometry / buffers	Create, allocate, bind, upload and build
Simulation	Declare capacities; bind both memory ranges
Pipeline	Set rigid state and optional liquid materials
Initialization list	Reset state; write bodies and constraints
Frame objects	Pool, buffers, timelines, queries and fence

Initialize geometry and simulation state before recording a frame. Make all initialization writes visible to the first physics step.

16.4 After submission

Wait or poll completion before reusing a command buffer or query slot. Read simulation status before accepting results after suspected overflow. Read timestamps with availability, using the queue's period and valid-bit count. A CPU readback additionally needs a device-to-host dependency and invalidation for noncoherent memory.

Do not call DeviceWaitIdle every frame to compensate for missing dependencies. Track the resources and completion values that each consumer actually uses.

Related sections: [14.2], [14.4], [5.4]

17 Fundamental declarations

Types and identifiers

17.1 Types, handles & constants

```
#define PM_API_VERSION_0_2 ((uint32_t)(2u << 12))
#define PM_WHOLE_SIZE UINT64_MAX
#define PM_TIMEOUT_INFINITE UINT64_MAX
#define PM_INVALID_INDEX UINT32_MAX
#define PM_QUEUE_FAMILY_IGNORED UINT32_MAX
#define PM_QUEUE_FAMILY_EXTERNAL (UINT32_MAX - 1u)
#define PM_FALSE 0u
#define PM_TRUE 1u
typedef uint32_t PmBool32;
typedef uint32_t PmFlags;
typedef uint64_t PmDeviceSize;
typedef uint64_t PmStageFlags;
typedef uint64_t PmAccessFlags;
typedef struct PmInstance_T* PmInstance;
typedef struct PmPhysicalDevice_T* PmPhysicalDevice;
typedef struct PmDevice_T* PmDevice;
typedef struct PmQueue_T* PmQueue;
typedef struct PmDeviceMemory_T* PmDeviceMemory;
typedef struct PmBuffer_T* PmBuffer;
typedef struct PmGeometry_T* PmGeometry;
typedef struct PmSimulation_T* PmSimulation;
typedef struct PmPhysicsPipeline_T* PmPhysicsPipeline;
typedef struct PmCommandPool_T* PmCommandPool;
typedef struct PmCommandBuffer_T* PmCommandBuffer;
typedef struct PmSemaphore_T* PmSemaphore;
typedef struct PmFence_T* PmFence;
typedef struct PmQueryPool_T* PmQueryPool;
```

Handles are opaque pointers, not structures applications may dereference. PMAPI_ATTR and PMAPI_CALL default to empty and may be supplied by the platform ABI. All declarations use C linkage in C++. PmBool32 is PM_FALSE or PM_TRUE. Each command has a PFN_pmCommandName function-pointer typedef with exactly the displayed return type, parameters and PMAPI_CALL convention.

17.2 Structure types

```
typedef enum PmStructureType {
    PM_STRUCTURE_TYPE_UNDEFINED = 0,
    PM_STRUCTURE_TYPE_INSTANCE_CREATE_INFO = 1,
    PM_STRUCTURE_TYPE_DEVICE_CREATE_INFO = 2,
    PM_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO = 3,
    PM_STRUCTURE_TYPE_BUFFER_CREATE_INFO = 4,
    PM_STRUCTURE_TYPE_GEOMETRY_CREATE_INFO = 5,
    PM_STRUCTURE_TYPE_SIMULATION_CREATE_INFO = 6,
    PM_STRUCTURE_TYPE_PHYSICS_PIPELINE_CREATE_INFO = 7,
    PM_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO = 8,
    PM_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO = 9,
    PM_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO = 10,
    PM_STRUCTURE_TYPE_SUBMIT_INFO = 11,
    PM_STRUCTURE_TYPE_NATIVE_DEVICE_IMPORT_INFO = 12,
    PM_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO = 13,
} PmStructureType;
```

Every extensible input structure sets sType to the identifier for its exact structure type. pNext is NULL unless a supported extension defines the chained structure. An identifier does not enable a feature or extension.

A Symbol index

PFN_pmVoidFunction — [3.1]
PmAccessFlags — [17.1]
pmAllocateCommandBuffers — [10.2]
PmAllocateFn — [3.3]
pmAllocateMemory — [5.3]
PmAllocationCallbacks — [3.3]
PmAxisLimit — [8.2]
PmAxisMotor — [8.2]

PmAxisSpring — [8.2]
pmBeginCommandBuffer — [10.2]
pmBindBufferMemory — [6.2]
pmBindGeometryMemory — [6.4]
pmBindSimulationMemory — [7.2]
PmBool32 — [17.1]
PmBuffer — [17.1]
PmBufferCopy — [11.1]
PmBufferCreateInfo — [6.1]
PmBufferMemoryBarrier — [12.2]
PmBufferRange — [6.2]
PmBufferView — [6.3]
pmCmdApplyRigidInputs — [7.5]
pmCmdBeginStatisticsQuery — [14.3]
pmCmdBindPhysicsPipeline — [10.5]
pmCmdBuildGeometry — [6.5]
pmCmdCopyBuffer — [11.1]
pmCmdCopyFluidParticles — [11.2]
pmCmdCopyRigidStates — [11.2]
pmCmdEndStatisticsQuery — [14.3]
pmCmdPipelineBarrier — [12.2]
pmCmdResetQueryPool — [14.1]
pmCmdResetSimulation — [10.4]
pmCmdSetFluidParticles — [10.4]
pmCmdSetRigidStates — [7.5]
pmCmdStepSimulation — [10.5]
pmCmdWriteRigidBodies — [7.4]
pmCmdWriteRigidConstraints — [8.3]
pmCmdWriteTimestamp — [14.2]
PmCommandBuffer — [17.1]
PmCommandBufferBeginInfo — [10.2]
PmCommandPool — [17.1]
PmCommandPoolCreateInfo — [10.1]
PmConstraintFrame — [8.1]
PmConstraintType — [8.1]
PmCouplingState — [9.3]
PmCouplingType — [9.3]
pmCreateBuffer — [6.1]
pmCreateCommandPool — [10.1]
pmCreateDevice — [4.5]
pmCreateFence — [12.5]
pmCreateGeometry — [6.4]
pmCreateInstance — [3.2]
pmCreatePhysicsPipeline — [9.4]
pmCreateQueryPool — [14.1]
pmCreateSemaphore — [12.3]
pmCreateSimulation — [7.1]
pmDestroyBuffer — [6.1]
pmDestroyCommandPool — [10.1]
pmDestroyDevice — [4.5]
pmDestroyFence — [12.5]
pmDestroyGeometry — [6.4]
pmDestroyInstance — [3.2]
pmDestroyPhysicsPipeline — [9.4]
pmDestroyQueryPool — [14.1]
pmDestroySemaphore — [12.3]
pmDestroySimulation — [7.1]
PmDevice — [17.1]
PmDeviceCreateInfo — [4.5]
PmDeviceMemory — [17.1]
PmDeviceQueueCreateInfo — [4.5]
PmDeviceSize — [17.1]
pmDeviceWaitIdle — [12.4]

pmEndCommandBuffer — [10.2]
pmEnumerateInstanceVersion — [3.2]
pmEnumeratePhysicalDevices — [4.1]
PmFence — [17.1]
PmFlags — [17.1]
PmFluidCapacity — [7.1]
pmFlushMappedMemoryRanges — [5.4]
PmFormat — [6.3]
PmFormatProperties — [15.2]
pmFreeCommandBuffers — [10.2]
PmFreeFn — [3.3]
pmFreeMemory — [5.3]
PmGeometry — [17.1]
PmGeometryCreateInfo — [6.4]
pmGetBufferMemoryRequirements — [6.2]
pmGetDeviceMemoryBudget — [14.5]
pmGetDeviceProcAddr — [3.1]
pmGetDeviceQueue — [4.5]
pmGetFenceStatus — [12.5]
pmGetGeometryMemoryRequirements — [6.4]
pmGetInstanceProcAddr — [3.1]
pmGetNativeBuffer — [13.3]
pmGetNativeDevice — [13.2]
pmGetNativeSemaphore — [13.4]
pmGetPhysicalDeviceFeatures — [4.4]
pmGetPhysicalDeviceFormatProperties — [15.2]
pmGetPhysicalDeviceInteropProperties — [13.1]
pmGetPhysicalDeviceLimits — [15.1]
pmGetPhysicalDeviceMemoryProperties — [5.1]
pmGetPhysicalDeviceProperties — [4.2]
pmGetPhysicalDeviceQueueFamilies — [4.3]
pmGetQueryPoolResults — [14.4]
pmGetSemaphoreCounterValue — [12.3]
pmGetSimulationMemoryRequirements — [7.2]
pmGetSimulationStatus — [14.3]
pmGetVulkanBufferEXT — [13.7]
pmGetVulkanDeviceEXT — [13.6]
pmGetVulkanSemaphoreEXT — [13.8]
pmImportNativeBuffer — [13.3]
pmImportNativeDevice — [13.2]
pmImportNativeSemaphore — [13.4]
pmImportVulkanBufferEXT — [13.7]
pmImportVulkanDeviceEXT — [13.6]
pmImportVulkanSemaphoreEXT — [13.8]
PmInstance — [17.1]
PmInstanceCreateInfo — [3.2]
pmInvalidateMappedMemoryRanges — [5.4]
PmLiquidMaterialState — [9.2]
pmMapMemory — [5.4]
PmMappedMemoryRange — [5.4]
PmMemoryAllocateInfo — [5.3]
PmMemoryBarrier — [12.2]
PmMemoryBudget — [14.5]
PmMemoryHeap — [5.1]
PmMemoryProperties — [5.1]
PmMemoryRequirements — [5.2]
PmMemoryType — [5.1]
PmMotionType — [7.3]
PmNativeBackend — [13.1]
PmNativeBufferInfo — [13.3]
PmNativeDeviceImportInfo — [13.2]
PmNativeDeviceInfo — [13.2]
PmNativeHandle — [13.1]

PmNativeInteropProperties — [13.1]
PmNativeQueueInfo — [13.2]
PmParticleState — [10.4]
PmPhysicalDevice — [17.1]
PmPhysicalDeviceFeatures — [4.4]
PmPhysicalDeviceLimits — [15.1]
PmPhysicalDeviceProperties — [4.2]
PmPhysicsPipeline — [17.1]
PmPhysicsPipelineCreateInfo — [9.4]
PmQueryPool — [17.1]
PmQueryPoolCreateInfo — [14.1]
PmQueryType — [14.1]
PmQueue — [17.1]
PmQueueFamilyProperties — [4.3]
pmQueueSubmit — [12.4]
pmQueueWaitIdle — [12.4]
pmResetCommandBuffer — [10.2]
pmResetCommandPool — [10.1]
pmResetFences — [12.5]
PmResult — [2.1]
PmRigidBodyInfo — [7.4]
PmRigidConstraintInfo — [8.3]
PmRigidInput — [7.5]
PmRigidSolverState — [9.1]
PmRigidState — [7.3]
PmSemaphore — [17.1]
PmSemaphoreCreateInfo — [12.3]
PmSemaphoreSubmitInfo — [12.3]
PmSimulation — [17.1]
PmSimulationCreateInfo — [7.1]
PmSimulationMemoryBinding — [7.2]
PmSimulationMemoryRequirements — [7.2]
PmStageFlags — [17.1]
PmStepInfo — [10.5]
PmStepStatistics — [14.3]
PmStructureType — [17.2]
PmSubmitInfo — [12.4]
pmUnmapMemory — [5.4]
PmVulkanBufferImportInfo — [13.7]
PmVulkanDeviceImportInfo — [13.6]
PmVulkanDeviceInfo — [13.6]
PmVulkanQueueInfo — [13.6]
pmWaitForFences — [12.5]
pmWaitSemaphore — [12.3]